- Endpoints Disponíveis
- ConexAI API – Documentação Técnica
  - Execução com Docker
    - 1. Build e execução simples
    - 2. Usando Docker Compose (recomendado)
  - Endpoints Disponíveis
    - GET /status
    - POST /chat/pergunta
    - GET /chat/historico?usuario=nome
  - Arquitetura e Componentes
    - ChatGestao
    - ValidadorPergunta
    - GeradorRespostaLLM
    - HistoricoInteracao
    - InterfaceTerminal
    - carregador_modelo.py
    - formatador_prompt.py
  - Suporte a Múltiplas Versões de Modelo
  - Informações Técnicas
  - Execução Local
  - Detalhes dos ends points
    - GET /status
    - POST /chat/pergunta
    - GET /chat/historico?usuario=weslei
  - Informações Técnicas
  - Fluxo Interno
  - Requisitos
- ConexAI V1
  - Identidade do Modelo
  - Treinamento
  - Exemplos de Uso
    - Entrada:
  - Comportamento Esperado
  - Armazenamento
  - Benchmark Interno
  - Observações
- ConexAI V2

    - Versão
    - Objetivo do Modelo
    - Pipeline de Treinamento
        - 1. Dados Utilizados
        - 2. Pré-processamento
        - 3. Modelagem com LoRA
        - 4. Configurações de Treinamento
    - Artefatos Gerados
    - Carregamento do Modelo
    - Teste de Inferência
    - Prompt de Identidade Incorporado
    - Resultados Observados
- Documentação Técnica - Módulo de Exportação de Dados Visuais
    - Objetivo
    - Formato do Arquivo Exportado
        - Exemplo 1: Gráfico de Linha (dados temporais)
        - Exemplo 2: Gráfico de Pizza ou Barra (categorias)
    - Localização do Arquivo
    - Observações Importantes

Endpoints Disponíveis

ConexAI API – Documentação Técnica

> Esta API expõe funcionalidades do modelo **ConexAI VX** via FastAPI, permitindo que aplicações externas (como o backend em Node.js) interajam com o modelo, salvem contexto e acessem históricos de conversa.

Execução com Docker

1. Build e execução simples

```
docker build -t conexai .
docker run -p 8000:8000 conexai
```

2. Usando Docker Compose (recomendado)

```
docker-compose up --build
```

> O serviço estará disponível em: `http://localhost:8000`

Você pode configurar:

- A versão do modelo com `VERSAO_MODELO` no `.env`
- O banco de dados com `DB_PATH`
- A porta com `API_PORT`

Endpoints Disponíveis

GET /status

Retorna o status da API e a versão do modelo carregado.

POST /chat/pergunta

Envia uma pergunta para o ConexAI. A resposta será gerada e registrada no histórico.

GET /chat/historico?usuario=nome

Lista os chats anteriores de um usuário específico.

Arquitetura e Componentes

ChatGestao

Classe orquestradora que gerencia o ciclo de vida de uma interação. Recebe as perguntas, valida com ValidadorPergunta, gera resposta com GeradorRespostaLLM e registra no histórico com HistoricoInteracao.

ValidadorPergunta

Valida a pergunta com base em filtros lexicais e semânticos, bloqueando termos técnicos e fora do escopo da IA.

GeradorRespostaLLM

Classe responsável por gerar respostas usando transformers.pipeline, com base no modelo carregado dinamicamente (V1, V2...).

HistoricoInteracao

Registra interações (pergunta, resposta, validade) em memória e no banco SQLite. Permite listar ou limpar registros associados a um chat_id.

InterfaceTerminal

Camada de interação com o terminal para testes locais. Substituível por interface web ou CLI.

carregador_modelo.py

Script que carrega dinamicamente o modelo, tokenizer e config, de acordo com a versão desejada (V1, V2, ...).

formatador_prompt.py

Contém funções utilitárias para montar o prompt de entrada com base em instruções padronizadas.

Suporte a Múltiplas Versões de Modelo

O ConexAI possui suporte a múltiplas versões:

- ConexAI V1: Baseado em deepseek-llm-7b-chat
- ConexAI V2: Baseado em mistralai/Mistral-7B-Instruct-v0.2

A seleção é feita dinamicamente no criar_chat(versao: str).

Informações Técnicas

- **Frameworks:** FastAPI, Transformers, PEFT (LoRA), SQLite
- **Quantização:** 4-bit (NF4 via BitsAndBytes)
- **Modelo carregado dinamicamente**
- **Tokenização:** baseada em AutoTokenizer

Execução Local

```
pip install -r requirements.txt
uvicorn main:app --reload
```

Detalhes dos ends points

GET /status

Descrição:

Retorna o status da API e a versão do modelo carregado.

Exemplo de Requisição:

```
GET http://localhost:8000/status
```

Resposta:

```
{
  "status": "API online",
  "modelo": "ConexAI V1"
}
```

POST /chat/pergunta

Descrição:

Envia uma pergunta para o ConexAI. A resposta será gerada com base nos filtros e contexto, e registrada no banco.

Requisição:

```
POST http://localhost:8000/chat/pergunta
Content-Type: application/json
```

Corpo JSON:

```
{
  "usuario": "weslei",
  "pergunta": "Qual é o faturamento da empresa?"
}
```

Resposta:

```
{
  "mensagem": "Pergunta processada."
}
```

> O modelo automaticamente cria um novo chat se o usuário ainda não tiver interagido ou se não houver `chat_id` ativo.

`GET /chat/historico?usuario=weslei`

Descrição:

Retorna a lista de chats existentes para o usuário informado.

Requisição:

```
GET http://localhost:8000/chat/historico?usuario=weslei
```

Resposta:

```
[
  {
    "id": 1,
    "nome": "Qual é o faturamento da empresa?",
    "data_criacao": "2024-04-16 13:05:02"
  },
  {
    "id": 2,
    "nome": "Como reduzir inadimplência?",
    "data_criacao": "2024-04-16 13:15:40"
  }
]
```

Informações Técnicas

- **Modelo:** `deepseek-ai/deepseek-llm-7b-chat` com LoRA (`ConexAI V1`)
- **Quantização:** 4-bit via `BitsAndBytes` (NF4)
- **Frameworks:** FastAPI + Transformers + PEFT + SQLite

- **Banco de dados:** `contexto_chat.db` (local, extensível para outros SGBDs)
- **Tokenização:** baseada em `AutoTokenizer` (pad_token = eos_token)

Fluxo Interno

1. **Validação semântica:** `ValidadorPergunta` filtra perguntas fora de escopo técnico.
2. **Persistência:** Contexto é salvo automaticamente no SQLite.
3. **Histórico:** Chats são organizados por usuário e título (título = 1ª pergunta).
4. **IA:** Usa LoRA aplicada ao modelo base, com geração controlada (`max_tokens`, `top_p`, `temperature`, etc).

Requisitos

- Python 3.10+
- Instale dependências com:

```
pip install -r requirements.txt
```

- Rode a API com:

```
uvicorn main:app --reload
```

ConexAI V1

Identidade do Modelo

Nome público: ConexAI V1 **Descrição:** Assistente treinado para compreender métricas empresariais, extratos, cobranças e pagamentos com linguagem natural. **Base técnica:** `deepseek-ai/deepseek-llm-7b-chat` **Frameworks:** Transformers, PEFT (LoRA), Datasets, TRL **Quantização:** 4-bit (BitsAndBytes - NF4)

Treinamento

Tipo: Ajuste fino com LoRA (Low-Rank Adaptation) **Técnica usada:** SFT (`Supervised Fine-Tuning`) **Tokens de entrada:** `<|im_start|>` e `<|im_end|>` (formato tipo ChatML) **Épocas:** 3 **Batch Size:** 2 **Gradient Accumulation Steps:** 2 **Dataset:** Dados empresariais reais (anonimizados), extraídos dos sistemas internos da Conexa (extrato, pagamento e cobrança). **Total de exemplos:** *X* exemplos (*editar com o número real*) **Tempo de treino estimado:** Aproximadamente *Y* horas (*editar se possível*)

Exemplos de Uso

Entrada:

```
<|im_start|>user
Qual o valor do extrato da conta Banco do Brasil?<|im_end|>
<|im_start|>assistant
O saldo atual da conta Banco do Brasil é R$4.959,27<|im_end|>
```

Comportamento Esperado

- **Identidade:** Quando questionado, o modelo se apresenta como "ConexAI V1"
- **Respostas curtas e diretas**, com linguagem acessível e contextualizada
- Capacidade de **responder perguntas financeiras simples** sobre contas, extratos e cobranças
- Formato de resposta estruturado para facilitar compreensão humana e automação

---

Armazenamento

```
/modelos_conexai/conexai-v1/
│
├── adapter_model.bin              # Pesos do LoRA ajustado
├── config.json                    # Metadados do modelo
├── special_tokens_map.json
├── tokenizer_config.json
└── tokenizer/                     # Arquivos de tokenização
```

Benchmark Interno

Métrica	Valor Aproximado
Tempo de Resposta (T4)	~45-60s/token completion
Tokens usados	4-bit quantized (NF4)
Tempo de ajuste	~X horas (*editar*)
Performance subjetiva	Boa precisão nas queries sobre pagamentos e extrato

---

Observações

- O modelo ainda depende de otimizações no tempo de resposta.
- Planeja-se uma futura migração para `Mistral 7B` ou `LLaMA 3` com refinamentos maiores e mais responsivos.
- Versão gerenciada via `ConexAI Version Control`.

ConexAI V2

Versão

ConexAI V2

Modelo Base : `mistralai/Mistral-7B-Instruct-v0.2`

Framework : Hugging Face Transformers + PEFT (LoRA) + TRL (SFTTrainer)

Tipo de Modelo : Language Model ajustado para domínio financeiro-empresarial

Tarefa : Geração de linguagem natural (Causal Language Modeling)

Objetivo do Modelo

Adaptar um modelo de linguagem open-source para atuar como um **assistente financeiro virtual** com foco em:

- KPIs financeiros e operacionais
- Contexto empresarial
- Recomendações de gestão
- Atendimento baseado em dados de extrato, pagamento e cobrança

---

Pipeline de Treinamento

1. Dados Utilizados

- Arquivos reais da plataforma Conexa:
  - `Conexa - Admin Extrato.xls`
  - `Conexa - Admin Pagamento.xls`
  - `Conexa - Admin Cobranca.xls`

2. Pré-processamento

- Extração e normalização de colunas
- Conversão em prompts estruturados com marcações no estilo:

```
<|im_start|>user
[PERGUNTA]
<|im_end|>
<|im_start|>assistant
[RESPOSTA]
<|im_end|>
```

3. Modelagem com LoRA

- Modelo base: `mistralai/Mistral-7B-Instruct-v0.1`
- Estratégia: LoRA (Low-Rank Adaptation) via PEFT
- Hiperparâmetros LoRA:
  - `r = 8`
  - `lora_alpha = 32`
  - `lora_dropout = 0.05`
  - `target_modules = ["q_proj", "v_proj"]`

4. Configurações de Treinamento

- Batch size efetivo: 4 (2 por device × 2 acumulações)
- `num_train_epochs = 3`
- `learning_rate = 2e-5`
- `optimizer = paged_adamw_8bit`
- `fp16 = True`
- Salvo em: `modelos_conexAI/ConexAI V2`

Artefatos Gerados

```
modelos_conexAI/
└── ConexAI V2/
    ├── adapter_model.safetensors
    ├── adapter_config.json
    ├── tokenizer_config.json
    ├── tokenizer.json
```

Carregamento do Modelo

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
import torch
```

```
base_model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.1",
    device_map="auto",
    torch_dtype=torch.float16,
    trust_remote_code=True
)

model = PeftModel.from_pretrained(base_model, "modelos_conexAI/ConexAI V2")
model = model.merge_and_unload()

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-
v0.1")
```

Teste de Inferência

```
prompt = "<|im_start|>user\nO que são KPIs?
<|im_end|>\n<|im_start|>assistant\n"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

outputs = model.generate(
    **inputs,
    max_new_tokens=512,
    do_sample=True,
    temperature=0.7,
    top_p=0.9,
    pad_token_id=tokenizer.eos_token_id
)

resposta = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

Prompt de Identidade Incorporado

- Instrução no pipeline/chat:

  "Você é o ConexAI V2, um assistente financeiro virtual treinado para fornecer respostas com precisão e foco em KPIs empresariais."

Resultados Observados

- Tempo médio de resposta: ~45s a 1min com T4 15GB RAM
- Coerência temática: Alta
- Fluência: Alta
- Tempo de resposta: *ainda em processo de otimização*

Documentação Técnica - Módulo de Exportação de Dados Visuais

Objetivo

Este módulo é responsável por analisar a resposta textual da IA (ConexAI) e, quando possível, extrair dados estruturados adequados para visualizações gráficas (linha, barra ou pizza). Os dados são exportados no formato JSON para que o front-end possa renderizar os gráficos dinamicamente.Funcionamento Interno

- A resposta da IA é processada por expressões regulares.
- O sistema tenta identificar padrões de:
  - Categoria + Valor → ex.: 'Marketing: R$ 5000'
  - Data + Valor → ex.: 'Jan/2024 - R$ 12.000'
- Se os dados forem reconhecidos, são convertidos em uma estrutura JSON e salvos localmente.

Formato do Arquivo Exportado

O JSON gerado segue um dos formatos abaixo:

Exemplo 1: Gráfico de Linha (dados temporais)

{ "grafico": "linha", "titulo": "Evolução temporal extraída", "dados": [{ "data": "2024-01", "valor": 10000.0 }, { "data": "2024-02", "valor": 15000.0 }, { "data": "2024-03", "valor": 12500.0 }], "moeda": "BRL" }

Exemplo 2: Gráfico de Pizza ou Barra (categorias)

{ "grafico": "pizza", "titulo": "Distribuição extraída", "dados": [{ "categoria": "Marketing", "valor": 5000.0 }, { "categoria": "Operações", "valor": 7000.0 }, { "categoria": "TI", "valor": 3000.0 }], "moeda": "BRL" }

Localização do Arquivo

O JSON gerado é salvo localmente no projeto backend com o nome:

resposta_estruturada.json

Este arquivo pode ser:

- Lido diretamente pelo front-end (se compartilhado via rota/endpoint).
- Encaminhado para visualização em tempo real (via WebSocket/API REST futura).

Observações Importantes

- A IA não gera os gráficos — ela responde normalmente e o módulo tenta extrair dados dela.
- Se nenhum dado estruturado for detectado, nenhum JSON é gerado.
- A moeda padrão é "BRL".
- Ideal para visualizações com dados financeiros, KPIs por mês, departamento ou categoria.